

Disease Symptom Prediction using Retrieval-Augmented Generation (RAG)

AIO Vietnam - Module 1 Project Proposal

June 6, 2026

1 Project Goal

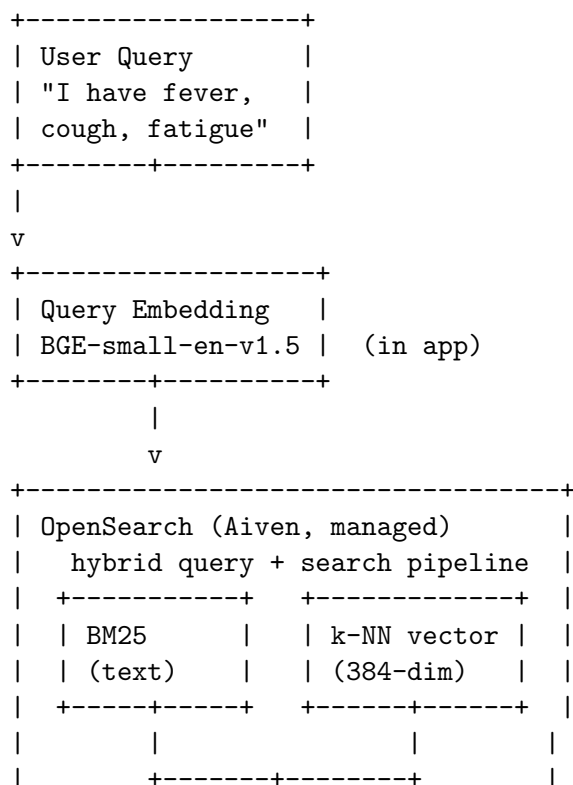
Build a lightweight and low-cost RAG system that accepts a user's symptom query and returns:

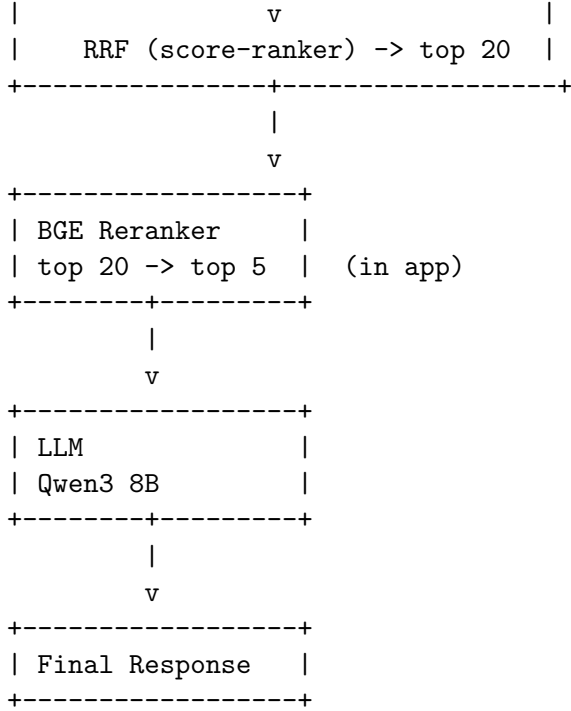
- Possible diseases
- Disease descriptions
- Severity information
- Recommended precautions

The objective is educational and demonstrational rather than clinical diagnosis.

2 System Architecture

The proposed MVP architecture is shown below:





3 Data Preparation

Each disease is stored as a single document. The source data follows the classic disease-symptom dataset format:

```

{
  "disease": "Influenza",
  "symptoms": ["fever", "cough", "fatigue"],
  "severity": "medium",
  "description": "...",
  "precautions": "..."
}

```

3.1 Three Logical Views of a Document

The key insight for *symptom-to-disease* retrieval is that the user query is a list of symptoms, so the searchable text must be symptom-dense. The description and precautions are mostly *output* fields, not *search* fields. Each record is therefore projected into three distinct views:

1. **Embedded text** (used to compute the dense vector stored in the `embedding knn_vector` field): symptom-first natural language.
2. **Keyword text** (indexed as an analyzed `text` field for BM25): raw symptom tokens, the disease name, and synonyms.
3. **Metadata payload** (returned in `_source`, not searched): severity, description, precautions, and provenance.

All three views live in a single OpenSearch document, so there is one source of truth and no cross-store synchronization.

3.2 Embedded Text Template

A symptom-first template keeps the embedding centroid close to symptom queries:

```
def build_embed_text(rec: dict) -> str:
    symptoms = ", ".join(rec["symptoms"])
    return (
        f"Disease: {rec['disease']}. "
        f"Symptoms: {symptoms}. "
        f"{rec['description']}"
    )
```

3.3 Symptom Normalization

The raw dataset uses underscores (`skin_rash`, `high_fever`). These are converted to natural language (`skin rash`, `high fever`) before embedding, because the embedding model handles natural phrases far better than `snake_case` tokens.

3.4 Query-Time Synonym Expansion

A small synonym map is applied to the user query (e.g. “tired” → “fatigue”, “throwing up” → “vomiting”). At this dataset scale, this improves recall more than swapping the embedding model.

3.5 Document Fields (OpenSearch Mapping)

Everything needed to render the final answer is stored in the document `_source`, so source data is never re-parsed after retrieval. Severity and precautions are deliberately kept *out* of the embedded text to avoid adding noise to similarity; severity is more useful as a filter (`keyword` type) or a post-retrieval field.

Field	OpenSearch type	Purpose
<code>doc_id</code>	keyword	stable id / provenance join
<code>disease</code>	text + keyword	display and grouping
<code>symptoms</code>	keyword[]	reranking and explanation
<code>severity</code>	keyword	display and filtering
<code>description</code>	text	LLM context
<code>precautions</code>	keyword[]	LLM context
<code>keyword_text</code>	text	BM25 lexical field
<code>embedding</code>	knn_vector (384)	dense vector (cosine)
<code>source</code>	keyword	provenance / citation

4 Embedding Model

4.1 Recommendation

For Module 1, use:

- BGE-small-en-v1.5

Advantages:

- Fast inference
- Small memory footprint

- Easy deployment
- Good retrieval quality

4.2 Embedding Strategy

Three practices materially improve retrieval quality with BGE:

- **Asymmetric query prefix.** BGE is trained with an instruction prefix. Queries are embedded as “Represent this sentence for searching relevant passages: ” + query, while documents are embedded *without* the prefix.
- **Normalized vectors + cosine similarity.** Embeddings are L2-normalized and the `knn_vector` field uses the `cosinesimil` space.
- **Exact / approximate search at MVP scale.** The corpus is on the order of tens of diseases, so exact k-NN is more than adequate; OpenSearch’s HNSW approximate index only matters once the corpus grows much larger.

The embedding is computed in the application (the embedding model is not hosted inside OpenSearch); the resulting 384-dimensional vector is sent into the `knn` sub-query at search time and stored in the `embedding` field at index time.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-small-en-v1.5")

doc_emb = model.encode(doc_texts, normalize_embeddings=True)
query_emb = model.encode(
    [query], normalize_embeddings=True,
    prompt="Represent this sentence for searching relevant passages: ",
)
```

4.3 Future Upgrade

Possible future embeddings:

- BioBERT
- PubMedBERT

5 Search Engine

A single managed **OpenSearch** service handles both retrieval modes: BM25 lexical search on analyzed `text` fields and dense vector search via the bundled k-NN plugin. This removes the need for a separate vector database – one engine, one source of truth, and no cross-store synchronization.

Advantages:

- BM25 lexical search and k-NN dense vector search in one engine
- Server-side hybrid fusion (including RRF) via a search pipeline, so no application-level merge code is required
- Metadata filtering (e.g. `severity`) in the same query
- Available on the Aiven free tier (4 GB RAM, 20 GB storage), so the MVP is fully managed at no cost

5.1 Managed Deployment (Aiven)

OpenSearch runs as a managed Aiven service from the start, so there is no container or volume to operate and the seeded index persists across runs. Connection credentials are provided to the application via environment variables / secrets and are never committed to source control. The free tier is a single node (no high availability), which is acceptable for the MVP; a paid plan adds replicas for production.

5.2 Index Mapping

The index defines the lexical field, the k-NN vector field, and the metadata fields returned in `_source`:

```
PUT /diseases
{
  "settings": { "index": { "knn": true } },
  "mappings": {
    "properties": {
      "doc_id":      { "type": "keyword" },
      "disease":     { "type": "text",
                      "fields": { "raw": { "type": "keyword" } } },
      "symptoms":    { "type": "keyword" },
      "severity":    { "type": "keyword" },
      "description": { "type": "text" },
      "precautions": { "type": "keyword" },
      "keyword_text": { "type": "text" },
      "embedding": {
        "type": "knn_vector",
        "dimension": 384,
        "space_type": "cosinesimil"
      }
    }
  }
}
```

5.3 Idempotent Ingestion

Documents are indexed with a deterministic id (`_id = doc_id`), so re-running ingestion upserts rather than duplicates – the load is idempotent and safe to repeat:

```
POST /diseases/_doc/influenza      # _id = doc_id => upsert, no duplicates
{
  "doc_id": "influenza",
  "disease": "Influenza",
  "symptoms": ["fever", "cough", "fatigue"],
  "severity": "medium",
  "description": "...",
  "precautions": ["rest", "fluids"],
  "keyword_text": "influenza fever cough fatigue ...",
  "embedding": [0.013, -0.052, ...] # 384-dim BGE vector (app-computed)
}
```

6 Retrieval Strategy

A hybrid retrieval approach is recommended.

6.1 BM25 Retrieval

A standard `match` query on the analyzed `keyword_text` field, scored with OpenSearch's default Okapi BM25.

Advantages:

- Exact keyword matching
- Effective for symptom names

6.2 Vector Retrieval (k-NN)

A `knn` query on the `embedding` field using cosine similarity.

Advantages:

- Semantic search
- Handles paraphrases and synonyms

6.3 Hybrid Search Pipeline

Both retrievers run inside OpenSearch within a single `hybrid` query, and their result sets are fused server-side by a search pipeline using **Reciprocal Rank Fusion (RRF)** (the `score-ranker-processor`). RRF is preferred over weighted score mixing because BM25 scores are unbounded and corpus-dependent while cosine similarity is bounded in $[-1, 1]$; RRF uses only ranks, so no per-dataset weight calibration is required.

$$\text{score}(d) = \sum_{r \in \{\text{bm25}, \text{knn}\}} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60$$

```
query
|-- BM25 (match)          -> ranked list --|
|-- k-NN (cosine)         -> ranked list --|
                                v
                                RRF (score-ranker processor)
                                v
                                top 20 docs
```

Define the fusion pipeline once, then attach it at search time:

```
PUT /_search/pipeline/hybrid-rrf
{
  "phase_results_processors": [
    { "score-ranker-processor": { "combination": { "technique": "rrf" } } }
  ]
}

GET /diseases/_search?search_pipeline=hybrid-rrf
{
  "size": 20,
  "query": {
    "hybrid": {
      "queries": [
        { "match": { "keyword_text": "fever cough fatigue" } },
        { "knn": { "embedding": { "vector": [/* 384-dim */], "k": 20 } } }
      ]
    }
  }
}
```

```
}  
}
```

The application then reranks these 20 candidates with `bge-reranker-base` and passes the top 5 to the LLM.

7 Reranking

Use:

- `bge-reranker-base`

Purpose:

- Improve relevance ranking
- Filter noisy retrieval results
- Select top disease candidates

Pipeline:

```
Top 20 Documents  
|  
v  
BGE Reranker  
|  
v  
Top 5 Documents
```

8 Large Language Model

8.1 Recommended Model

Qwen3 8B

Reasons:

- Open-source
- Low deployment cost
- Good reasoning capability
- Suitable for local inference

8.2 LLM Responsibilities

The LLM should:

- Summarize retrieved information
- Explain disease similarities
- Present possible diseases
- Recommend precautions
- Express uncertainty when appropriate

The LLM should not be treated as a medical diagnostic system.

9 Technology Stack

Component	Technology
Embedding	BGE-small-en-v1.5 (384-dim)
Search Engine	OpenSearch (Aiven, managed)
Vector Search	OpenSearch k-NN (cosine)
Keyword Search	OpenSearch BM25
Fusion	RRF (score-ranker search pipeline)
Reranker	bge-reranker-base
LLM	Qwen3 8B
Framework	LangChain / LlamaIndex (optional)

10 Scalability Roadmap

10.1 Phase 1 (MVP)

- OpenSearch on the Aiven free tier (BM25 + k-NN)
- Hybrid retrieval with server-side RRF fusion
- BGE-small-en-v1.5 embeddings
- bge-reranker-base
- Qwen3 8B

10.2 Phase 2 (Production)

Because OpenSearch is used from the start, scaling is mostly a matter of moving to a larger plan and tuning the same engine rather than migrating systems:

- Paid OpenSearch plan with replicas (high availability) and HNSW tuning for larger corpora
- BioBERT or PubMedBERT embeddings
- Larger reranker models
- GPT-5 or larger open-source LLMs
- Monitoring and feedback loops

11 Conclusion

For Module 1, the following architecture provides the best balance between simplicity, cost, and scalability:

```
User Query
|
v
Embedding (BGE-small, in app)
|
v
OpenSearch hybrid: BM25 + k-NN  ->  RRF (top 20)
|
```


v
Reranker (bge-reranker-base, top 5)
|
v
Qwen3 8B
|
v
Response

This architecture is easy to implement, inexpensive to operate, and can be scaled incrementally as project requirements grow.